

P3 Member Playbook

Reliable Data Transfer over UDP · CS-30003 Coding Assignment 1 · Revised 14 September 2026 for the post-mid-semester sprint

What changed, and why. Mid-semester exams took 1–14 September for every student, so no work happened in the original weeks 1 and 2. We are recovering it with a **two-week sprint, Monday 14 to Sunday 27 September**, covering the original weeks 1–5. The following week, **28 September to 5 October**, is a buffer for the long experiment runs and anything that slipped. From **6 October** we are back on the original dates, unchanged.

How to use this document. Everyone reads the sprint calendar (page 2), the gates, and Parts 0 and 1 today. After that, work from your own section. Every block tells you what to do, how you know you are finished, and what you must be able to say out loud about it. That last part matters most: the viva is worth 5 of the 10 marks and it is assessed *individually*.

Who is doing what

MEMBER	ROLE	OWNS	EXPERIMENT & REPORT SECTION
Shaili Seth 2405901 · shailiseth24	M1	Wire format, RFC 1071 checksum, metadata handshake, Stop-and-Wait, sender/receiver CLIs, FIN teardown, shared integrity suite	Exp 1 — goodput vs loss rate Report: wire format & session
Sinjan Mishra 2405910 · Thesin-01	M2	Timer heap, Jacobson/Karels RTO with Karn, Go-Back-N, adaptive/fixed RTO switch	Exp 4 — RTO sensitivity Report: timers, RTO, GBN
Sohini Pandit 2405913 · sohinix	M3	Selective Repeat, receiver buffer, selective ACKs, wraparound correctness, SACK stretch	Exp 2 — goodput vs window Report: Selective Repeat
Sohan Mandal 2405912 · sohan1611	M4	Channel emulator, capacity calibration, experiment harness, statistics, plots, tc/netem validation	Exp 3 — reordering Report: methodology & emulator

Why the sprint is possible

Compressing five weeks into two only works because of three things, and the plan leans on all of them. **First**, the two components everything depends on — `Packet` and the emulator — already exist as drafts with 33 passing tests, so their rewrites are days, not weeks. **Second**, several tasks do not depend on anything else and have been moved to the front: Sinjan's timer heap and RTT estimator, and Sohini's receiver buffer, are pure data structures that can be built and tested before `Packet` lands. **Third**, experiments start on Stop-and-Wait and Go-Back-N data while Selective Repeat is still being finished.

The sprint is also exactly when copy-pasting gets tempting. Straight after exams, with a tight deadline, pasting a chatbot's Go-Back-N will feel reasonable. It is not: the viva is individual, sir will ask you to change your code live, and a chatbot gives every group the same answer to a textbook problem — which the brief treats as collusion. Use AI to understand. Type the code yourself.

Sprint calendar — 14 to 27 September

Bold entries are merges or deliverables other people are waiting on. Green rows are gates.

DATE	SHAILI (M1)	SINJAN (M2)	SOHINI (M3)	SOHAN (M4)
SPRINT A · FOUNDATIONS · covers original weeks 1–3				
Mon 14	Clone, set identity, build green	Clone, set identity, build green	Clone, set identity, build green	Rewrite <code>ChannelConfig</code> + <code>TraceLog</code>
Tue 15	Read header spec + RFC 1071; first commit	Read RFC 6298 + GBN chapter; first commit	Read SR theory; first commit	Rewrite <code>Channel</code>
Tue night	GATE 1 — all four cloned, identity verified, first commit pushed. Contributor list shows four names.			
Wed 16	Rewrite <code>Packet</code>	<code>TimerWheel</code> design + start	Window-bound note; start receiver buffer	Finish <code>Channel</code> ; rewrite <code>NetEm</code>
Thu 17	<code>Packet</code> merged, 13/13	<code>TimerWheel</code> + tests	Receiver buffer + tests	Emulator merged, 20/20
Thu night	GATE 2 — <code>Packet</code> and emulator both rewritten by hand and merged; both <code>AI-USE.md</code> boxes ticked.			
Fri 18	Metadata handshake	<code>TimerWheel</code> merged	Wraparound test cases; buffer merged	JDK in WSL2; calibrate on WSL2
Sat 19	<code>StopAndWait</code>	<code>RttEstimator</code> + hand-worked tests	SR design: per-packet timers	<code>RunStats</code> to CSV
Sun 20	CLIs; S&W at 0% loss	<code>RttEstimator</code> merged	Start <code>SelectiveRepeat</code>	Help S&W run through emulator
Sun night	GATE 3 — timer heap, RTT estimator and receiver buffer merged; 8 MiB Stop-and-Wait transfer matches at 0% loss; WSL2 calibration committed.			
SPRINT B · PROTOCOLS AND EVIDENCE · covers original weeks 4–5				
Mon 21	S&W at 10% loss × 10 seeds	<code>GoBackN</code>	<code>SelectiveRepeat</code> core	<code>run_matrix.py</code>
Tue 22	FIN / FINACK teardown	<code>GoBackN</code> continues	SR core; fallback decision tonight	Lock configs; first plot
Wed 23	SHA-256 check; integrity suite skeleton	<code>GoBackN</code> merged, 5×10	Finish SR; wraparound; legal vs illegal window	<code>plots.py</code> with 95% CI
Wed night	GATE 4 — Go-Back-N passes 5 profiles × 10 seeds; Stop-and-Wait matches at 10% loss × 10 seeds; a plot exists that a script generated.			
Thu 24	Integrity suite covers all three	Adaptive RTO into all three	SR merged	Check operating points; help SR
Thu night	GATE 5 · CORE COMPLETE — all three protocols pass one shared integrity suite.			
Fri 25	Exp 1: S&W + GBN cells	<code>--rto fixed:X</code> switch	Exp 2 start	Exp 3; launch overnight runs
Sat 26	Exp 1: SR cells	Support experiment runs	Exp 2 runs	Exp 3; monitor runs
Sun 27	Figure 1	Exp 4 design note	Figure 2	Figure 3
Sun night	GATE 6 · SPRINT DONE — figures 1–3 generated by script from a first pass of at least 3 seeds per cell.			
BUFFER · 28 SEP – 5 OCT · then original schedule from 6 October				

After the sprint — back on the original dates

DATES	SHAILI (M1)	SINJAN (M2)	SOHINI (M3)	SOHAN (M4)
Sep 28 – Oct 5 buffer	Exp 1 at full 10 seeds; methodology write-up with Sohan	Anything that slipped; start report section	Exp 2 at full 10 seeds; anything that slipped	Drive full 10-seed runs; methodology write-up
Oct 6–12 week 6	Cross-teaching; report section	Exp 4 with spurious labelling	Cross-teaching; report section	tc netem validation; cross-teaching
Oct 13–19 week 7	Report review	Report section done	Stretch: SACK blocks, <i>only if Core is solid</i>	Final figures; report assembly
Oct 20–26 week 8	Code freeze 22 October · final report · viva drills — everyone			

Part 0 · Setup — everyone, by Tuesday 15 September

The repository folder is not shared. Sohan's copy lives on Sohan's laptop. Git is distributed: **each of you makes your own complete copy on your own laptop** by cloning it from GitHub, then works, commits and pushes from there. GitHub is only the meeting point. Every command below runs on *your* machine.

1. Install the tools on your laptop

Install **Git** and **JDK 21** (Adoptium Temurin). On Windows, also set up WSL2, because the experiments must run on Linux:

```
wsl --install -d Ubuntu
wsl -d Ubuntu
sudo apt update && sudo apt install -y openjdk-21-jdk make python3-pip
java -version          # must print 21.x
```

2. Clone the repository onto your laptop

On Windows open **Git Bash**; on Mac open Terminal. Go to wherever you keep projects, then:

```
git clone https://github.com/sohan1611/rdt-udp.git
cd rdt-udp
```

You now have your own `rdt-udp` folder. Everything from here happens inside it.

3. Set your git identity — inside that folder

```
git config user.name "Your Full Name"
git config user.email "your-github-email@example.com"
```

Use an email that is on your GitHub account, or you will not count as a contributor. Check it at github.com/settings/emails. If you would rather not use a personal address, that page also shows a private one ending in `@users.noreply.github.com`, which links correctly. A commit with an unrecognised email shows a grey avatar and does not appear on your profile or in the contributor list. Sir treats commit history as evidence in individual moderation, so get this right once.

4. Check the build

```
./build.sh          # expect 33 tests, 0 failed
```

5. Make your first commit — by Tuesday night

Open `AI-USE.md`, put your name against your role heading (replace `__[name]_`), and push it on a branch:

```
git checkout -b setup/your-name
git add AI-USE.md
git commit -m "Add my name to AI-USE.md"
git push -u origin setup/your-name
git log -1 --format='%an <%ae>'      # confirm the name and email are yours
```

Then open a pull request on GitHub. The first push asks you to sign in; on Windows a browser window opens, which is normal. Once this merges, you appear in the contributor list.

6. Read, in this order

- `README.md` — what the project is (10 min)
- `docs/PLAN.md` sections 1–4 — the design decisions we committed to (30 min)
- `AI-USE.md` — the AI rules and the two open rewrite items
- Your own section of this document

Part 1 · Rules for the sprint

Daily check-in — every night by 10 pm, in the group chat

Three lines, no more: **done today**, **doing tomorrow**, **blocked on**. In a two-week sprint a silent day costs half a day for whoever is waiting on you. If you are blocked, say so the same evening, not at the next gate.

Branches and pull requests

Never commit straight to `main`. One branch per task: `m1/packet-rewrite`, `m2/timer-wheel`, `m3/receiver-buffer`, `m4/emulator-rewrite`.

```
git checkout main && git pull          # always start from the latest main
git checkout -b m2/timer-wheel
# ... work, commit as you go ...
git push -u origin m2/timer-wheel      # then open the PR on GitHub
```

Review pairs, and a 12-hour turnaround

WHEN	PAIRS
Sprint A · Sep 14–20	Shaili ↔ Sinjan · Sohini ↔ Sohan
Sprint B · Sep 21–27	Shaili ↔ Sohini · Sinjan ↔ Sohan

Review within 12 hours. In a sprint an unreviewed PR blocks someone. If your pair is stuck, anyone else may review instead.

The explanation check still applies, even in a hurry. Before approving, the reviewer asks two or three *why* questions. *Why this data structure? What happens if this ACK is lost? Why this bound?* If the author cannot answer, it does not merge. A five-minute voice call is fine. It rehearses the viva every day and it catches copy-pasted code while there is still time.

Commits

- **At least one commit every working day** during the sprint, under your own name and email.
- Never commit on someone else's behalf — the brief counts that as non-contribution by the absent member.
- **No Co-Authored-By lines, no bot pull requests, Dependabot stays off.** The contributor list must show exactly our four names.

AI and libraries

Permitted: explaining concepts, debugging, reviewing your code, generating tests, boilerplate, plotting scripts, improving writing. **Not permitted:** generating the core protocol implementation and submitting it as your own. Log material help in `AI-USE.md` the same day. **Banned:** Netty, Aeron, KryoNet, JGroups, any QUIC library, and TCP anywhere on the data path.

Shaili Seth · M1 · Framing and Session

You own the envelope every packet travels in and the simplest of the three protocols. **Thursday 17 is the most important date in the whole sprint:** nobody can test a protocol until `Packet` is merged.

Sprint A · 14–20 September

Mon 14 – Tue 15

Setup and reading

1. Complete Part 0 in full. Push your first commit by Tuesday night.
2. Read `docs/PLAN.md` section 4.1 and study the 20-byte header diagram until you can draw it from memory.
3. Read **RFC 1071**, section 1 — about one page, and it describes the checksum.

Done when: your first commit is merged and you can sketch the header layout without looking.

Wed 16 – Thu 17

Rewrite `Packet.java` · BLOCKS EVERYONE

1. `git checkout -b m1/packet-rewrite`
2. Open `src/main/java/rdt/Packet.java`. Read it once for the shape, then **delete the bodies** of `encode()`, `decode()` and `checksum()` and write them yourself from the diagram. If you cannot write them without looking back, go to the diagram — not to the old code.
3. Three traps, each worth a lost day if missed:
 - `seq` and `ack` are unsigned 32-bit; a Java `int` is signed. Read with `Integer.toUnsignedLong(buf.getInt())` into a `long`.
 - The 16-bit fields need `Short.toUnsignedInt(buf.getShort())`.
 - Compute the checksum with its own field set to **zero**, then write it into that field.
4. Run `./build.sh` until `PacketTest` reports 13 of 13.
5. Open the PR by Thursday afternoon. Sinjan reviews and asks three *why* questions.
6. Tick the `Packet.java` box in `AI-USE.md`, sign and date it.

Done when: merged into `main` by Thursday night with `PacketTest: 13 run, 0 failed`, and the `AI-USE.md` box ticked.

Be able to explain: why one's complement rather than a plain sum · what end-around carry does · why a single-bit flip is *always* caught · what breaks without `Integer.toUnsignedLong` · why the checksum field is zeroed first.

Fri 18

Metadata handshake

1. A `seq=0` DATA packet whose payload carries the filename, file size and SHA-256 hash. The receiver parses it and replies with ACK 0. Real data then starts at `seq=1`.

Done when: the receiver prints the filename, size and hash it received, and they match what the sender sent.

Sat 19 – Sun 20

Stop-and-Wait and the CLIs

1. Write `StopAndWait.java` against the `ArqProtocol` interface. Sender: send one packet, start a timer, wait for its ACK, retransmit on timeout. Receiver: verify the checksum, ACK, deliver in order, silently discard duplicates.
2. Use **alternating-bit** sequencing (0, 1, 0, 1...).
3. Write `app/Sender.java` and `app/Receiver.java` taking a file, host, port and `--protocol` flag.
4. Test through Sohan's emulator at 0% loss:

```
java -cp build/classes emulator.NetEm --listen 9000 --to 127.0.0.1:9001 --seed 1 --both delay=20 &
java -cp build/classes app.Receiver --port 9001 --out received.bin &
java -cp build/classes app.Sender --file test8m.bin --to 127.0.0.1:9000 --protocol stopwait
sha256sum test8m.bin received.bin      # the two hashes must match
```

Done when: an 8 MiB file crosses the emulator at 0% loss and both hashes match. This is part of Gate 3 on Sunday night.

Be able to explain: what happens when an ACK is lost but the data arrived · why one alternating bit is enough here but not for Go-Back-N · why Stop-and-Wait throughput is capped at MSS/RTT however fast the link is.

Sprint B · 21–27 September

Mon 21 – Tue 22

Harden Stop-and-Wait; teardown

1. Run Stop-and-Wait at **10% loss across 10 seeds** and fix whatever breaks.
2. Add **FIN / FINACK** so a transfer ends cleanly rather than by the receiver guessing from silence. No fixed sleeps or timeout hacks anywhere in shutdown.

Done when: 10 of 10 seeds at 10% loss match, each ending with an explicit FIN exchange.

Wed 23 – Thu 24

The shared integrity suite

1. Add the end-of-transfer SHA-256 check against the hash from the handshake.
2. Wednesday: build the integrity suite skeleton running Stop-and-Wait and Go-Back-N over several channel profiles and seeds.
3. Thursday: add Selective Repeat as soon as Sohini merges it. **This suite decides Core complete**, so it has to be ready before SR lands, not after.

Done when: the suite runs 3 protocols × 5 profiles × 10 seeds and every transfer matches — Gate 5, Thursday night.

Fri 25 – Sun 27

Experiment 1 — goodput vs loss rate

1. Friday: run the Stop-and-Wait and Go-Back-N cells at window 32 across loss rates 0, 0.1, 0.5, 1, 2, 5, 10, 20%. Stop-and-Wait is slow at high loss — start these first and let them run overnight.
2. Saturday: add the Selective Repeat cells.
3. Sunday: generate **Figure 1** through Sohan's `plots.py`. At least 3 seeds per cell this week; the full 10 run in the buffer week.

Done when: Figure 1 is generated by script from committed CSVs, one line per protocol, with error bars.

After the sprint

Sep 28 – Oct 26

Buffer, then original weeks 6–8

1. **Sep 28 – Oct 5 (buffer):** Experiment 1 at the full 10 seeds; write the methodology section with Sohan.
2. **Oct 6–12:** teach your module in the cross-teaching session and learn the others; draft your report section on the wire format and session layer.
3. **Oct 13–19:** review the full report draft; check every caption says what to notice.
4. **Oct 20–26:** viva drills. Code freeze 22 October.

Sinjan Mishra · M2 · Timers and Go-Back-N

You own everything to do with time: when to stop waiting, how long to wait, and what to resend. Your sprint is ordered deliberately — the timer heap and RTT estimator come first because **neither depends on Packet**, so you are never blocked on anyone in Sprint A.

Sprint A · 14–20 September

Mon 14 – Tue 15

Setup and reading

1. Complete Part 0. First commit pushed by Tuesday night.
2. Read **RFC 6298** — short, and exactly the Jacobson/Karels algorithm you will build.
3. Read the Go-Back-N section of Kurose & Ross, chapter 3.

Done when: first commit merged, and you can say what SRTT and RTTVAR are.

Wed 16 – Fri 18

TimerWheel

1. Wednesday: a half-page design note in `docs/` — what a timer heap is, why a single thread rather than a timer per packet, how the socket receive timeout comes from the earliest deadline. Commit it; it becomes part of your report section.
2. Write `TimerWheel.java`: a `PriorityQueue` of deadlines from `System.nanoTime()`, with `schedule`, `cancel`, and `earliest-deadline` operations.
3. **Single-threaded.** The socket loop asks the heap how long until the next timer and blocks in `receive` for exactly that long. A timeout and an arriving ACK are both just events on one loop, so there is no race between them.
4. Do **not** use `ScheduledExecutorService` or `java.util.Timer`. They reintroduce exactly the concurrency the brief warns about.
5. Tests: ordering under many insertions, cancelling an already-fired timer, identical deadlines.
6. Wednesday evening: review Shaili's `Packet` PR and ask three *why* questions.

Done when: merged by Friday night, and the heap reports the correct earliest deadline after a randomised sequence of schedules and cancels. Sohini is waiting on this for Selective Repeat.

Be able to explain: how you avoid a race between a timeout firing and an ACK arriving — there is no race, and you should be able to say exactly why.

Sat 19 – Sun 20

RttEstimator

1. Implement Jacobson/Karels: $SRTT = (1-\alpha) \cdot SRTT + \alpha \cdot R$ with $\alpha = 1/8$; $RTTVAR = (1-\beta) \cdot RTTVAR + \beta \cdot |SRTT-R|$ with $\beta = 1/4$; $RTO = SRTT + 4 \cdot RTTVAR$, clamped to a sensible minimum.
2. Add **Karn's algorithm**: never sample RTT from a retransmitted packet — when an ACK arrives for a packet sent twice you cannot tell which copy it answers. On timeout, double the RTO instead, and resume sampling only after a clean, un-retransmitted exchange.
3. Test against values you work out by hand on paper. It is the fastest way to catch an α/β mix-up.

Done when: merged by Sunday night, matching your hand-worked numbers.

Be able to explain: what Karn's algorithm protects against, with a concrete example · why RTO adds $4 \times RTTVAR$ instead of using SRTT alone.

Sprint B · 21–27 September

Mon 21 – Wed 23

Go-Back-N

1. Everything you need is merged by now: `Packet`, the emulator, and your own heap.
2. Write `GoBackN.java`: a sliding send window of N packets, cumulative ACKs, one timer on the **base** of the window. On timeout, retransmit the base and everything after it.
3. Plug in your `RttEstimator` for the timeout value.
4. Run it through the integrity checks across 5 channel profiles and 10 seeds.

Done when: merged by Wednesday night, passing 5 profiles $\times$ 10 seeds — part of Gate 4.

Be able to explain: why Go-Back-N retransmits the whole window · why it needs $W \leq 2^k - 1$ sequence numbers · why it falls behind Selective Repeat as loss rises.

Thu 24 – Fri 25

Adaptive and fixed RTO everywhere

1. Thursday: wire `RttEstimator` into all three protocols.
2. Friday: add a fixed-RTO switch for Experiment 4, supporting multipliers of 1.2, 1.5, 2 and 3.

Done when: every protocol runs with either `--rto adaptive` or `--rto fixed:1.5`.

Sat 26 – Sun 27

Support runs; plan Experiment 4

1. Help keep the overnight experiment runs healthy.
2. Write a one-page design note for Experiment 4: how Sohan's JSONL trace tells you whether each retransmitted packet's original really arrived, and therefore whether the retransmission was spurious or necessary.

After the sprint

Sep 28 – Oct 26

Buffer, then original weeks 6–8

1. **Sep 28 – Oct 5 (buffer):** finish anything that slipped; start your report section.
2. **Oct 6–12:** run **Experiment 4** — fixed RTO at 1.2/1.5/2/3 $\times$ against adaptive, labelling each retransmission spurious or necessary from the trace. This is the experiment no other group will have.
3. **Oct 13–19:** finish your report section on timers, RTO and Go-Back-N.
4. **Oct 20–26:** viva drills. Code freeze 22 October.

Sohini Pandit · M3 · Selective Repeat

You own the hardest module, and the sprint gives it the least slack of anything. Two things make that workable: your receiver buffer is built as a **standalone data structure in Sprint A**, before any network code, and there is a **fallback decision on Tuesday 22** so the project cannot stall on one module.

Sprint A · 14–20 September

Mon 14 – Tue 15

Setup and theory

1. Complete Part 0. First commit pushed by Tuesday night.
2. Read the Selective Repeat section of Kurose & Ross, chapter 3, including the example where the sender's and receiver's windows disagree.
3. Work out on paper why Selective Repeat needs $W \leq 2^k - 1$ while Go-Back-N needs only $W \leq 2^k$. Draw the failure: a small sequence space, a full window, every ACK lost, and a retransmission the receiver mistakes for new data.

Done when: first commit merged and you can draw that failure case from memory.

Be able to explain: the window bound with its concrete failure case, not just the formula. It is the single most likely question you will get.

Wed 16 – Fri 18

Receiver buffer, standalone

1. Wednesday: commit your window-bound explanation as a short note in `docs/`. It opens your report section.
2. Build the **receiver buffer** with no sockets at all: it accepts packets in any order, holds out-of-order ones, and releases a contiguous run the moment a gap fills.
3. Test it as a pure data structure: feed 1, 3, 4, 2 and assert it emits 1, then nothing, then nothing, then 2, 3, 4 together. Add duplicates.
4. Friday: write the **wraparound test cases**. Make the sequence space size configurable so a test can force a wrap with a tiny space.
5. Review Sohan's emulator PR (Wednesday or Thursday) with three *why* questions.

Done when: the buffer is merged by Friday night, passing out-of-order, duplicate and wraparound tests with no network code involved.

Sat 19 – Sun 20

Design and start SelectiveRepeat

1. Saturday: design the sender side on paper — **each unacknowledged packet has its own timer**, held in Sinjan's `TimerWheel` (merged Friday).
2. Sunday: create `SelectiveRepeat.java` and wire your merged buffer into the receiver side.

Sprint B · 21–27 September

Mon 21 – Tue 22

SelectiveRepeat core · fallback decision

1. Sender window with per-packet timers; receiver buffering out-of-order arrivals and acknowledging each packet individually.
2. On timeout, retransmit **only** that packet — the whole point of the protocol.
3. Encode selective ACKs in the `sackCount` header field, already reserved, so no wire-format change is needed.

Tuesday 22, 10 pm — fallback decision, with Sohan. If Selective Repeat does not transfer a file correctly at 10% loss by Tuesday night, switch to a **single timer on the oldest unacknowledged packet** instead of one per packet. It is still correct and still beats Go-Back-N, and it is far simpler to finish by Thursday. Document it as a deliberate simplification; per-packet timers move to week 7. Taking this on Tuesday is good planning. Leaving it to Thursday is what breaks the sprint.

Done when: a file transfers at 10% loss and the logs show only lost packets being resent, not whole windows — or the fallback has been chosen.

Wed 23 – Thu 24

Finish — Core complete depends on you

1. Make the wraparound tests from Friday 18 pass.
2. Prove the window bound experimentally: with a small sequence space, a **legal** window transfers correctly and an **illegal** one corrupts the file. Most groups never test this.
3. Merge by Thursday afternoon so Shaili can add you to the integrity suite that evening.

Done when: Selective Repeat passes the shared integrity suite alongside the other two — **Gate 5, Core complete**, Thursday night.

Be able to explain: why Selective Repeat needs per-packet timers where Go-Back-N needs one · how the receiver decides whether an arriving packet is new or a retransmission · what your legal-versus-illegal window test shows.

Fri 25 – Sun 27

Experiment 2 — goodput vs window size

1. Go-Back-N against Selective Repeat across windows 1, 2, 4, 8, 16, 32, 64, 128, at 1% and 5% loss.
2. This is the experiment that shows the gap widening with window size — the heart of our claim. At least 3 seeds per cell this week.
3. Sunday: generate **Figure 2**.

Done when: Figure 2 is generated by script from committed CSVs.

After the sprint

Sep 28 – Oct 26

Buffer, then original weeks 6–8

1. **Sep 28 – Oct 5 (buffer):** Experiment 2 at the full 10 seeds; finish anything that slipped.
2. **Oct 6–12:** teach Selective Repeat to the other three; draft your report section.
3. **Oct 13–19:** stretch — SACK blocks in the live protocol, *only* if Core is solid. If the Tuesday 22 fallback was taken, restore per-packet timers here instead.
4. **Oct 20–26:** viva drills. Code freeze 22 October.

Sohan Mandal · M4 · Channel and Evidence

You own the fake bad network everyone tests against and all the measurement that turns working code into marks. **Thursday 17 matters as much for you as for Shaili:** Stop-and-Wait cannot be tested until your emulator rewrite is merged. You are already set up, so you start today.

Sprint A · 14–20 September

Mon 14 – Thu 17

Rewrite the emulator by hand · BLOCKS SHAILI

1. The catalogue entry *requires* it: "a channel emulator **you write yourself**." The draft is AI-written, so it must be replaced. The 20 tests stay — they tell you the rewrite is correct.
2. `git checkout -b m4/emulator-rewrite`, then rewrite in this order:
 - **Mon 14:** `ChannelConfig.java` and `TraceLog.java` — small, gets you moving.
 - **Tue 15 – Wed 16:** `Channel.java` — the impairment model, the real work.
 - **Wed 16 – Thu 17:** `NetEm.java` — the socket loop and release queue.
3. Know these three decisions before you write them, because you will defend them:
 - **Separate process**, so the protocol has no test-only branches.
 - `java.util.Random(seed)`, whose algorithm is fixed by the Java specification, so a seed reproduces on any machine. Never `ThreadLocalRandom` OR `SecureRandom`.
 - **A fixed number of random draws per packet**, so changing loss does not shift the delay sequence, and two cells of a sweep differ only in the variable under test.
4. PR by Thursday afternoon; Sohini reviews. Tick your `AI-USE.md` box.

```
./build.sh      # ChannelTest 14/14 and NetEmSmokeTest 6/6 must pass on your own code
```

Done when: merged by Thursday night with all 20 emulator tests passing on code you wrote, and the `AI-USE.md` box ticked — Gate 2.

Be able to explain: why a separate process · why `java.util.Random` · what the fixed draw count buys · why the release queue needs a tiebreak, and what reordering would appear without one.

Fri 18

Calibrate on the real experiment host

1. Install JDK 21 inside WSL2 (Part 0, step 1).
2. Run the calibration there and commit the result, replacing the Windows figure. The experiments run on Linux, so that is the ceiling that counts.

```
java -Xms512m -Xmx512m -cp build/classes emulator.Calibrate
git add results/calibration.csv
```

Done when: `results/calibration.csv` records a WSL2 run and the ceiling is noted in the group chat.

Sat 19 – Sun 20

RunStats; unblock Stop-and-Wait

1. Write `RunStats.java`: per-run counters — goodput, wire throughput, retransmissions, timeouts, duplicate ACKs — emitted as one CSV row.
2. Sunday: sit with Shaili while Stop-and-Wait first runs through the emulator. Most first-run problems are addressing and port mix-ups that take minutes with both of you looking.

Done when: a Stop-and-Wait run produces one CSV row with sensible numbers.

Sprint B · 21–27 September

Mon 21 – Wed 23

The evidence pipeline

1. **Mon 21:** `analysis/run_matrix.py` — reads a config, runs the sweep across parameters and seeds, and appends rows to `results/*.csv`.
2. **Tue 22:** lock the configs in `analysis/configs/` (8 MiB file, 1400-byte payload); produce the **first plot** from real Stop-and-Wait data. End to end beats pretty. Join Sohini's fallback decision at 10 pm.
3. **Wed 23:** `analysis/plots.py` with mean and 95% confidence intervals over seeds. Every figure generated by script; none made by hand.

Done when: one command runs a sweep and a second turns its CSV into a PNG, with no manual steps — part of Gate 4.

Thu 24

Check the numbers; help land Core complete

1. Confirm every planned operating point sits below **half** the WSL2 ceiling.
2. Help Sohini get Selective Repeat merged. Thursday night is Core complete, and she carries the hardest module. Help her understand and debug — do not write it for her.

Fri 25 – Sun 27

Experiment 3; run the matrix

1. **Friday: launch the overnight runs.** Stop-and-Wait at high loss can take minutes per run; the full matrix is hours. Starting on Sunday would sink the gate.
2. Run **Experiment 3** yourself: retransmission count against reordering probability 0, 1, 2, 5, 10, 20% for all three protocols.
3. Keep Shaili's and Sohini's runs healthy; restart anything that dies.
4. Sunday: generate **Figure 3**; confirm figures 1–3 all regenerate from committed CSVs.

Done when: figures 1–3 regenerate by script from first-pass data of at least 3 seeds per cell — Gate 6, sprint done.

After the sprint

Sep 28 – Oct 26

Buffer, then original weeks 6–8

1. **Sep 28 – Oct 5 (buffer):** drive the full 10-seed runs for experiments 1–3; write the methodology section with Shaili.
2. **Oct 6–12:** `tc netem` cross-validation — re-run the loss sweep on Linux's emulator and show the curves agree within confidence intervals, explaining any divergence. Run the cross-teaching session, and make sure *you* can explain Selective Repeat.
3. **Oct 13–19:** final figures; assemble the report from everyone's sections.
4. **Oct 20–26:** confirm `make all` reproduces every figure from a clean clone. Viva drills. Code freeze 22 October.

Gates at a glance

Check each one in the group chat on the night it falls. If one is missed, say so that night and decide what moves — the buffer week exists to absorb exactly this, but only if slips are raised immediately.

WHEN	GATE	MUST BE TRUE	IF MISSED
Tue 15, night	1 · Setup	All four cloned, identity verified, first commit pushed; contributor list shows four names	Fix on Wednesday morning; nothing else can proceed
Thu 17, night	2 · Foundations	Packet and emulator rewritten by hand, merged; both <code>AI-USE.md</code> boxes ticked	Shaili and Sohan pair on Friday
Sun 20, night	3 · Sprint A	Timer heap, RTT estimator, receiver buffer merged; 8 MiB Stop-and-Wait matches at 0% loss; WSL2 calibration committed	Slip goes to Mon 21; Sprint B tightens
Tue 22, night	SR decision	Selective Repeat transfers correctly at 10% loss — or the single-timer fallback is chosen	Take the fallback
Wed 23, night	4 · Protocols	Go-Back-N passes 5×10 ; Stop-and-Wait matches at 10% loss $\times 10$ seeds; a script-generated plot exists	Raise that night
Thu 24, night	5 · Core complete	All three protocols pass one shared integrity suite	Friday, then buffer week
Sun 27, night	6 · Sprint done	Figures 1–3 by script from first-pass runs, ≥ 3 seeds per cell	Finish in buffer week
Oct 5	Buffer	Experiments 1–3 at full 10 seeds; methodology drafted	—
Oct 12	Week 6	Figure 4 done; netem validation done; everyone can trace a packet through code they did not write	—
Oct 19	Week 7	Full report draft read and annotated by all four	—
Oct 26	Week 8	make all reproduces every figure from a clean clone	—

Viva preparation

Five of the ten marks are awarded to you individually, and sir will ask you to modify your code live. In week 8, run these at each other until nobody hesitates — several are about code you did not write.

TOPIC	QUESTION
Checksum	Explain the Internet checksum byte by byte. Why one's complement? Why end-around carry? Why is a single-bit flip always caught?
Unsigned fields	Why <code>Integer.toUnsignedLong</code> on the sequence field? Show what breaks without it.
Windows	Why does Selective Repeat need $W \leq 2^{k-1}$ when Go-Back-N needs $W \leq 2^k - 1$? Draw the failure case.
Timers	How do you avoid a race between a timeout firing and an ACK arriving?
Karn	What is Karn's algorithm protecting against? Give a concrete example.
Loss	Walk through what happens when an ACK is lost but the data arrived.
Reproducibility	How do you know two runs with the same seed are identical? Why <code>java.util.Random</code> ?
Measurement	How do you know your goodput curve reflects the protocol and not your machine running out of CPU?
Live change	Change the window from 32 to 8 while a transfer runs. Add a new packet type, now.

Still to ask sir. The brief we have refers to a "Section 6" on individual moderation and a "Section 7" on scanning networks, but ends at Section 6, the proposal template — ask whether a fuller version exists. It also gives no proposal deadline or submission method anywhere.